Traduce Señas:

Manual técnico

Juan Mauricio Suaza Solórzano

Luis Alejandro Duarte Aldana

Juan Camilo Fierro Esquivel

Instructor: Karol Daniela Correa

4 de noviembre de 2026

Tabla de contenido

Introducción 4

Objetivo 4

1. Requisitos del sistema 4

2. Herramientas empleadas en el desarrollo 4

3. Arquitectura 5

    3.1 Servicios del backend 6

    3.2 Rutas de la API 7

4. Base de datos 10

    4.1 Una base por servicio 10

    4.2 Migraciones SQL vigentes 10

    4.3 Retención y borrado de datos 11

    4.4 Desajustes con la documentación 11

5. Seguridad 12

    5.1 Autenticación 12

    5.2 Verificación de correo y recuperación 12

    5.3 Autorización (RBAC) 13

    5.4 Riesgos abiertos 14

6. Reconocimiento de señas (IA) 15

7. Frontend 16

8. Diagramas 18

9. Pruebas 18

10. Instalación 19

10.1 Instalación local (resumen) 19

10.2 Instalación en servidor web 19

11. Ejecución del aplicativo 19

12. Real frente a documentado 20

13. Pendientes 20

Referencias 22

Introducción

Este manual describe la versión vigente del sistema: Backend `v1.4.0` (rama `main`, commit `a962982`) y Frontend `v1.3.1` (rama `main`, commit `a5e231d`). Para instalar el sistema, ver el Manual de instalación.

- `Backend@main:<ruta>` = archivo de la rama `main` del repositorio Backend. Se lee sin cambiar de rama con `git show origin/main:<ruta>` (dentro del repositorio Backend).
- `Frontend@main:<ruta>` = archivo de la rama `main` del repositorio Frontend.

Objetivo

Este manual explica a quien llega al proyecto cómo está construido Traduce Señas: herramientas, arquitectura de microservicios, rutas de la API, base de datos, seguridad, reconocimiento de señas, frontend, diagramas, pruebas e instalación.

1. Requisitos del sistema

Ver Manual de instalación, sección 2.1.

2. Herramientas empleadas en el desarrollo

Tabla 1

Herramientas empleadas en el desarrollo

Tema	Herramientas	Fuente
Backend	Node.js 20, Express 5 y TypeScript en todos los servicios y en el gateway. PostgreSQL 16 con `pg` (una base por servicio). MongoDB 7 con `mongodb` (muestras de gestos). RabbitMQ 3 con `amqplib` (eventos). `jsonwebtoken` para los JWT, `bcrypt` para las contraseñas, `zod` para validar los cuerpos de iam, `nodemailer` para el correo, `google-auth-library` para el inicio con Google, `express-rate-limit` y `http-proxy-middleware` en el gateway. Pruebas con `node:test`.	`Backend@main:services/*/package.json`, `packages/shared/package.json`, `scripts/run-tests.js`

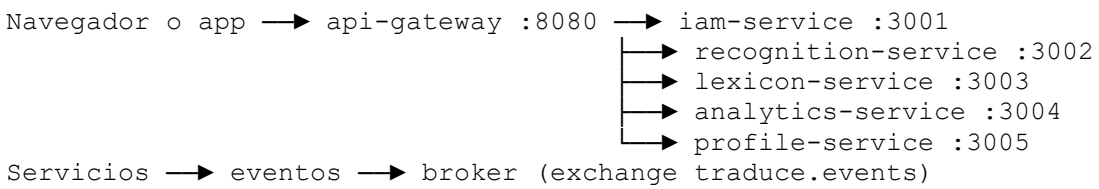
Tema	Herramientas	Fuente
Infraestructura	Docker y Docker Compose (13 contenedores en local). Caddy 2 como proxy con HTTPS en producción. Cada Dockerfile define un `HEALTHCHECK`.	`Backend@main:docker-compose.yml`, `docker-compose.prod.yml`, `deploy/Caddyfile`, `services/*/Dockerfile`
Frontend	Expo 57, React Native 0.86, react-native-web, React Navigation 6, axios, AsyncStorage y expo-secure-store para el token, expo-auth-session para Google, react-native-webview para el modo Palabras en el celular, TensorFlow.js 4 para el modelo propio, Jest 29 y Playwright 1.63. **Sin React Query.**	`Frontend@main:package.json`

3. Arquitectura

Tabla 2

Arquitectura

Tema	Estado	Fuente
Estilo general	**Microservicios**: un API gateway y cinco servicios (`iam`, `recognition`, `lexicon`, `analytics` y `profile`), cada uno con su proceso y su base de datos.	`Backend@main:docker-compose.yml`
Estilo interno	**Hexagonal** en cada servicio: carpetas `domain`, `application`, `ports` (inbound y outbound) y `adapters`. `packages/shared` solo comparte configuración, JWT, middleware HTTP, contratos de eventos y el bus de eventos.	`Backend@main:services/*/src`, `packages/shared/src`
Comunicación	HTTP desde el gateway hacia cada servicio y entre servicios por su nombre interno. Eventos asíncronos por el broker, en el exchange `traduce.events`, con una cola de mensajes fallidos.	`Backend@main:packages/shared/src/events`
Datos	Una base relacional por servicio (cinco en total) y una colección documental en `recognition`.	`Backend@main:services/*/db`, `docker-compose.yml`



En producción, Caddy recibe las peticiones por HTTPS y reenvía `/api/*` al gateway. Lo demás va a la web exportada. Los servicios 3001 a 3005 no se publican fuera de la red interna de Docker.

3.1 Servicios del backend

Tabla 3

Servicios del backend

Servicio	Puerto interno	Base de datos	Qué hace	Prefijos que atiende el gateway
`api-gateway`	8080	Ninguna	Punto de entrada único: CORS con lista blanca, límites de peticiones por IP, comprobación opcional del token, proxy y `/health` agregado.	Todos los de `/api`
`iam-service`	3001	`iam`	Registro, inicio de sesión, verificación de correo, recuperación y cambio de contraseña, inicio con Google y Facebook, eliminación de cuenta, roles y permisos.	`/api/auth`, `/api/users`, `/api/iam`
`recognition-service`	3002	`recognition` y MongoDB	Traducciones guardadas, plantillas de señas, muestras de gestos (solo administradores) y scripts de entrenamiento fuera del servidor.	`/api/translations`, `/api/recognition`, `/api/sign-templates`, `/api/samples`
`lexicon-service`	3003	`lexicon`	Catálogo de señas, alfabeto y medios (modelos 3D y miniaturas).	`/api/lexicon`
`analytics-service`	3004	`analytics`	Eventos de uso y reportes agregados por sección.	`/api/analytics`
`profile-service`	3005	`profile`	Perfil y preferencias, logros, retos diarios y puntos, notificaciones y reportes semanales y mensuales por correo a los administradores.	`/api/profile`, `/api/notifications`, `/api/achievements`

El gateway reenvía cada prefijo a su servicio sin leer el cuerpo de la petición. Comprueba el token de forma opcional: un token caducado o inválido no impide llegar a `/api/auth`, para que se pueda iniciar sesión otra vez. Aplica un límite general por IP y otro más estricto a las rutas

de autenticación sensibles (inicio de sesión, registro, códigos y recuperación). Responde `404` con el formato de error común si la ruta no existe.

Los servicios se comunican por eventos. Estos son los que define el paquete compartido:

- `iam` publica `iam.UserRegistered` e `iam.UserDeleted`. Los escribe primero en una tabla de salida (`outbox_events`) dentro de su propia base y un proceso los publica después. Así un evento no se pierde si el broker no está disponible en ese momento.
- `recognition` publica `recognition.TranslationProduced`.
- `profile` publica `profile.AchievementUnlocked` y `profile.PreferencesChanged`.
- `recognition`, `analytics` y `profile` consumen `iam.UserDeleted` para borrar los datos de la cuenta. Cada uno crea su cola al arrancar.

3.2 Rutas de la API

"Token" = cabecera `Authorization: Bearer <token>`. El permiso o rol indicado es el que comprueba la ruta; los errores de negocio siguen el formato `{ success, code, message }`.

Tabla 4

Rutas de la API

Método	Ruta	Token	Permiso o rol
POST	`/api/auth/register`	No	—
POST	`/api/auth/login`	No	—
POST	`/api/auth/verify-email`	No	— (exige código **y** contraseña)
POST	`/api/auth/verify-code`	No	—
POST	`/api/auth/resend-verification`	No	—
POST	`/api/auth/forgot-password`	No	—
POST	`/api/auth/reset-password`	No	—
POST	`/api/auth/google`	No	— (pide consentimiento si la cuenta es nueva)

Método	Ruta	Token	Permiso o rol
POST	`/api/auth/facebook`	No	— (pide consentimiento si la cuenta es nueva)
GET	`/api/users/me`	Sí	—
DELETE	`/api/users/me`	Sí	— (contraseña, o código por correo si la cuenta no tiene contraseña)
POST	`/api/users/me/delete-code`	Sí	—
POST	`/api/users/me/password`	Sí	—
GET	`/api/users/stats`	Sí	—
GET	`/api/users/admins`	Solo token de servicio	Permiso interno de directorio
GET	`/api/iam/me/access`	Sí	— (devuelve mis roles y permisos)
GET	`/api/iam/roles`	Sí	`users.manage`
POST	`/api/iam/users/:userId/roles`	Sí	`users.manage`
DELETE	`/api/iam/users/:userId/roles/:roleName`	Sí	`users.manage`
DELETE	`/api/iam/roles/:roleId`	Sí	`users.manage`
POST	`/api/translations`	Sí	—
GET	`/api/translations/history`	Sí	—
GET	`/api/translations/me/stats`	Sí	—
GET	`/api/translations/stats`	Sí	— (la ruta no exige rol; ver 5.4)
DELETE	`/api/translations/:id`	Sí	—
GET	`/api/sign-templates`	No	— (público a propósito)
POST	`/api/sign-templates`	Sí	`models.manage`
DELETE	`/api/sign-templates/:id`	Sí	`models.manage`
GET y POST	`/api/samples`	Sí	Rol ADMIN
GET	`/api/lexicon`, `/search`, `/alphabet`, `/categories`, `/:code`	No	—

Método	Ruta	Token	Permiso o rol
POST, PATCH, DELETE y publicación en `/api/lexicon` (categorías, señas, localizaciones y recursos)	Sí	Rol ADMIN	
POST	`/api/analytics/events`	Sí	—
GET	`/api/analytics/summary`, `/top-signs`, `/daily`	Sí	Rol ADMIN
GET	`/api/analytics/reports/sections`	Sí	`stats.read`
GET y PATCH	`/api/profile/me`	Sí	— (cambiar `notificationsEnabled` exige `notifications.manage`)
GET y PATCH	`/api/profile/preferences`	Sí	— (mismo caso que `/me`)
GET	`/api/profile/challenges/today`	Sí	—
GET	`/api/profile/challenges/rewards/summary`	Sí	—
GET	`/api/achievements`	Sí	—
POST	`/api/achievements/activity`	Sí	—
GET	`/api/notifications`	Sí	—
PATCH	`/api/notifications/:id/read`	Sí	—
GET	`/health` y `/api/health`	No	— (estado agregado de los servicios)

Notas:

- Las rutas de `iam` que tocan roles comprueban `users.manage` dentro del caso de uso.
- Los contratos OpenAPI por servicio están en `docs/07-api/contracts/openapi`. No se han contrastado uno a uno con el código en este manual.
- La ruta `GET /api/users/admins` la usa `profile` con su propio token de servicio; ningún usuario lo recibe.

4. Base de datos

4.1 Una base por servicio

Cada servicio es dueño de su base. No hay tablas compartidas ni consultas entre servicios: si un servicio necesita datos de otro, los pide por HTTP interno o los recibe por eventos. En total hay **33 tablas** en las cinco bases relacionales y **una colección** en la base documental de `recognition`, donde se guardan las muestras de gestos.

4.2 Migraciones SQL vigentes

Las migraciones están en `services/<servicio>/db/`. El compose las monta por nombre en `/docker-entrypoint-initdb.d` del contenedor de cada base. PostgreSQL las ejecuta en orden **solo cuando el volumen se crea por primera vez**.

Tabla 5

Migraciones que se montan al crear el volumen

Servicio	Archivos montados
`iam`	`000_users`, `003_iam_roles_permissions`, `005_auth_email_normalization_user_tokens`, `006_users_status_email_verification`, `009_outbox_events`, `010_users_password_nullable`, `011_account_deletion_token`, `012_outbox_claim_token`, `013_users_terms_version`
`recognition`	`001_translations`, `002_sign_templates`, `003_erased_users`, `004_sign_templates_word_kind`, `005_translation_retention`
`lexicon`	`003_lexicon` (incluye la semilla del alfabeto)
`analytics`	`001_usage_event`, `002_erased_users`
`profile`	`001_profile` (incluye los logros), `002_erased_users`, `003_ui_languages`, `004_achievement_catalog`, `005_obtainable_achievements`, `006_translation_milestones`, `007_functional_achievements`, `008_admin_reports`, `009_daily_challenges`, `010_challenge_reward_achievements`

Un archivo nuevo no hace nada hasta que se añade su línea en la sección `volumes` del servicio de base correspondiente en `docker-compose.yml`. Para aplicar cambios a una base que ya existe, ver el Manual de instalación, sección 1.2.

Los archivos `master.xml` y los changelogs de Liquibase que aparecen en `analytics-service`, `iam-service`, `lexicon-service` y `recognition-service` no los ejecuta Docker Compose. Ya no reproducen el esquema actual; los `.sql` son la fuente de verdad.

4.3 Retención y borrado de datos

- **Traducciones.** `recognition` borra de verdad las traducciones más antiguas que el plazo de retención. El plazo se configura con `RECOGNITION_TRANSLATION_RETENTION_DAYS`; con `0` no se borra nada. Antes de borrar, los conteos diarios y por palabra pasan a `translation_daily_counts` y `translation_user_words`, de modo que las estadísticas no pierden historia.
- **Cuenta.** Al borrar la cuenta, `iam` elimina el usuario y sus códigos, y deja `iam.UserDeleted` en su tabla de salida. Cada servicio consumidor borra sus propios datos del usuario y guarda el identificador en `erased_users`. Así un evento que llega tarde no vuelve a crear datos de una cuenta borrada.
- **Último administrador.** La cuenta del único ADMIN no se puede borrar (código `LAST_ADMIN`).

4.4 Desajustes con la documentación

- El modelo de referencia de `docs/06-data` puede no coincidir con las migraciones. Las migraciones son la fuente de verdad.
- La estrategia de migración documentada (`node-pg-migrate`) no se usa. Las migraciones son SQL escrito a mano y se aplican con Docker Compose o a mano en una base existente.
- El ADR-003 habla de PostgreSQL y MongoDB en un solo almacén. La separación actual por servicio corresponde al ADR-008.

5. Seguridad

5.1 Autenticación

- **Token JWT** firmado con el secreto compartido `JWT_SECRET`. `iam` lo emite al iniciar sesión y los demás servicios lo verifican con el mismo secreto. El secreto debe tener al menos 32 caracteres: el arranque falla si no cumple.
- El token lleva el identificador del usuario (`sub`), el correo, sus roles y sus permisos. Caduca en un plazo fijo y **no hay renovación**: no existe `refresh` ni `logout` en la API.
- Los servicios comprueban la firma, el emisor y la audiencia. No consultan a `iam` en cada petición.
- **Contraseñas** con bcrypt (Provos y Mazières, 1999). El inicio de sesión compara siempre contra un hash: si no existe la cuenta usa un hash falso, para no revelar por el tiempo de respuesta si el correo existe. El estado de la cuenta (inactiva o bloqueada) solo se muestra después de comprobar la contraseña.
- **Inicio con Google y Facebook.** El backend verifica el token del proveedor antes de crear o abrir la sesión: con Google, mediante la verificación del ID token; con Facebook, mediante la comprobación del access token en la Graph API. Si la cuenta es nueva, el backend exige el consentimiento de los términos.

5.2 Verificación de correo y recuperación

- Cuenta nueva = `INACTIVE`. Se envía un **código de 6 dígitos** generado con `crypto.randomInt` y guardado como hash bcrypt. Caduca a los 15 minutos, admite 5 intentos, exige 60 s entre envíos y permite 5 envíos por hora.
- `verify-email` exige el código **y** la contraseña. Así otra persona no puede activar una cuenta ajena con solo el código.

- El registro exige la aceptación de los términos y registra su versión (`termsVersion`), como consentimiento de la Ley 1581 de 2012.
- Si el correo no se puede enviar, el registro conserva la cuenta y el usuario puede pedir un código nuevo.
- Para borrar una cuenta sin contraseña se usa un código por correo con su propio tipo de token.

5.3 Autorización (RBAC)

- Hay tres roles base: **USER**, **LINGUIST** y **ADMIN**, y 13 permisos (`packages/shared/src/security/permissions.ts`). Cada cuenta nueva recibe USER al registrarse.
- Los roles y permisos viajan **dentro del token**. Un cambio no surte efecto hasta que el usuario vuelve a iniciar sesión, y tras quitar un rol el token conserva sus permisos hasta que caduque. Para cortar todas las sesiones a la vez hay que cambiar `JWT_SECRET`.
- `requirePermission` comprueba un permiso del token. `requireRole` comprueba un rol del token.
- Reglas en `iam`: los roles base no se borran, un rol asignado no se borra, y el último ADMIN no puede perder su rol (código `LAST_ADMIN`).
- El primer ADMIN se asigna a mano con una orden SQL, porque no existe un endpoint para ello. Ver el Manual del administrador (`docs/14-training/admin-manual.md`), sección 3.
- Las muestras de gestos son datos biométricos: todo el recurso `/api/samples` exige rol ADMIN.

- Las analíticas se guardan como agregados por sección. El informe por sección exige `stats.read`.

5.4 Riesgos abiertos

Tabla 6

Riesgos abiertos

Riesgo	Gravedad	Detalle
Token vigente tras bloquear la cuenta	Media	El middleware compartido (`packages/shared/src/http/auth_middleware.ts`) solo verifica la firma. Una cuenta bloqueada conserva su token hasta que caduca.
Estadísticas globales de traducciones para cualquier cuenta	Media	`GET /api/translations/stats` exige solo token. Conviene limitarla a `stats.read` si los datos globales no deben verlos todos los usuarios.
Contraseña sin regla de fortaleza en el servidor	Media	El servidor solo limita el tamaño (72 bytes, límite de bcrypt). La regla de 8 caracteres y de tres de cuatro tipos (mayúscula, minúscula, número y símbolo) está solo en el cliente (`passwordStrength.ts`).
Límite de intentos solo por IP	Baja/Media	El gateway limita por IP. No se ha verificado un bloqueo temporal por cuenta tras varios intentos fallidos.
Token en el almacenamiento web	Media	En web el token se guarda en AsyncStorage, que usa `localStorage`: un script inyectado (XSS) puede leerlo. En el celular se usa almacenamiento seguro.
Credenciales en ramas anteriores	Alta	Hubo una contraseña en claro en documentación de una rama anterior. No se copia en este manual. Debe rotarse y retirarse de esa rama.
Textos legales del cliente sin contrastar	Media	Los textos de privacidad del frontend no se han comparado con el funcionamiento real. No se reproducen aquí.

Ya no son riesgos respecto a versiones anteriores: la validación de `JWT_SECRET` al arrancar, el control de rol en las plantillas (`models.manage`), la lista blanca de CORS, el límite de peticiones del gateway y la eliminación de cuenta, que sí borra los datos.

6. Reconocimiento de señas (IA)

- **El reconocimiento corre en el navegador** (en la web y dentro de la WebView de la app Android). `recognition-service` solo guarda y reparte plantillas y muestras: no infiere (`services/recognition-service/README.md`).
- Motor de visión: **MediaPipe Tasks Vision 0.10.14**, cargado por CDN, con HandLandmarker y PoseLandmarker (Lugaresi et al., 2019; Zhang et al., 2020). Extrae 21 puntos por mano y los normaliza.
- Dos modos en `TranslationScreen`. **Palabras** es el modo por defecto. **Abecedario** es el segundo.
  - **Palabras:** si existe el modelo propio (`public/models/lsc/`, TensorFlow.js, entrada de 30 cuadros por 128 valores y `glosas.json` con 54 palabras), lo usa (`wordEngine.web.ts`). Si no, usa el `GestureRecognizer` genérico de 7 gestos.
  - **Abecedario:** geometría de la mano, clasificador KNN (Cover y Hart, 1967) y comparación temporal DTW (Sakoe y Chiba, 1978) contra plantillas de 16 cuadros. Las plantillas se bajan de `GET /api/sign-templates?kind=motion`.
- Coexisten dos formatos de entrada (16 cuadros por 126 valores en DTW y 30 cuadros por 128 valores en el modelo TensorFlow.js). No son el mismo camino de cálculo.
- **En el celular:** el modo Palabras se abre en una WebView que carga la web en modo embebido (`?embed=word-engine`). El modo Abecedario no reconoce con la cámara en el celular: muestra un aviso.
- **Entrenamiento fuera del servidor** (`services/recognition-service/training/`): scripts TypeScript (`npm run ia:*`) para descargar y convertir el conjunto LSC-54 (Mora-Zarate et al., 2025), y subir plantillas. Los scripts de Python y los cuadernos de

Colab entrenan un modelo Conv1D con GRU y lo exportan a TensorFlow.js (Google, s. f.). El modelo resultante se copia a `Frontend/public/models/lsc/`.

- **Licencia del conjunto LSC-54:** CC BY-NC 4.0 (uso no comercial, con cita). Esto choca con la decisión abierta sobre una cuota comercial.
- El modelo aprendió de personas grabadas en un estudio y puede fallar con otra cámara. La pantalla de muestras (`ModelSamplesScreen`, solo para administradores) sirve para grabar repeticiones y reajustar el modelo. **La exactitud del modelo actual no está medida** en este manual.

7. Frontend

Estructura (`src` del repositorio Frontend): `app/` (configuración, proveedores y rutas), `feature/` (Admin, Alphabet, History, Profile, Stats, Translation y auth), `shared/` (componentes, constantes, servicios, tipos y utilidades) y `public/models/lsc/` (modelo).

- **Proveedores** (`App.tsx`): el estado de sesión lo gestiona `AuthProvider`, y los permisos, `AccessProvider`.
- **AccessContext** (`app/providers/AccessContext.tsx`): al iniciar sesión llama a `GET /api/iam/me/access` y guarda los roles y permisos. Si la llamada falla, deniega todo y avisa. `adminMode` equivale a `users.manage`, `canStats` a `stats.read` y `canAdmin` a `users.manage` o `models.manage`.
- **Navegación** (`MainTabNavigator.tsx`): con `adminMode` se muestran Estadística, Admin y Perfil. Sin él, Traducción, Alfabeto, Historial y Perfil. Un administrador no ve la pestaña de Traducción.

- **Cliente HTTP** (`shared/services/api.client.ts`): axios con el token en `expo-secure-store` (celular) o AsyncStorage (web). Un interceptor añade `Authorization` solo a las llamadas a la API propia y avisa ante un 401.
- **Dirección de la API** (`app/config/api.config.ts`): por defecto apunta a `https://traducesenas.tech/api`. La variable `EXPO_PUBLIC_API_URL` la cambia (por ejemplo, `http://localhost:8080/api` para el backend local). La web de producción usa la ruta relativa `/api`.
- **i18n** (`app/config/i18n/locales/`): español, inglés, francés y portugués. El idioma por defecto es el español.
- **Archivos `.web.ts`:** Metro elige `*.web.ts` en la web (por ejemplo, `wordEngine.web.ts`).

  Estado de conexión de las pantallas con el backend:

Tabla 7

Estado de conexión del frontend

Pantalla o función	¿Conectada?
Login, registro, verificar correo y reenviar código	Sí
Inicio con Google y Facebook	Sí, si la plataforma tiene su credencial configurada. Si falta, el botón avisa. Google no está disponible en Expo Go.
Olvidé mi contraseña, código y nueva contraseña	Sí (`forgot-password`, `verify-code` y `reset-password`)
Cambiar contraseña	Sí (`POST /api/users/me/password`). Si la cuenta no tiene contraseña, crea una con el flujo de código.
Eliminar cuenta	Sí (`DELETE /api/users/me`, con código por correo o contraseña)
Traducción (guardar) e Historial (listar y borrar)	Sí. El historial pide hasta 100 traducciones.
Permisos (`/api/iam/me/access`)	Sí
Plantillas de señas (`/api/sign-templates`)	Sí

Pantalla o función	¿Conectada?
Retos diarios, logros y notificaciones	Sí
Estadística	Sí (`translations/stats`, `users/stats` y `analytics/reports/sections`)
Gestión de roles (`/api/iam/roles` y relacionadas)	**No hay pantalla**: solo API

8. Diagramas

Los diagramas de clases vigentes están en `docs/08-uml/diagrams` (exportados a SVG, con la fuente PlantUML en `source/`). El índice está en `docs/08-uml/diagram-index.md`. El diagrama de casos de uso está en el Documento de análisis del software (evidencia 02).

Las figuras de clases que tenía la versión anterior de este manual describían el monolito y se han retirado. El alcance del MVP es solo de seña a texto o voz.

9. Pruebas

- **Backend.** `npm test` compila `@traduce/shared` y ejecuta todos los archivos `*.test.ts` de `services/*/tests` y `packages/shared/tests` con `node:test` (`scripts/run-tests.js`). Resultado del 4 de noviembre de 2026: **918 pruebas pasan, 0 fallan y 13 se omiten** porque requieren PostgreSQL. Las pruebas con base real se activan con las variables `PROFILE_TEST_DATABASE_URL`, `PROFILE_PREFERENCES_PG_TESTS` y `GAMIFICATION_PG_TESTS`.
- **Frontend.** `npm test` ejecuta Jest y el resultado del 4 de noviembre de 2026 fue **469 de 469**. `npm run typecheck` revisa los tipos. `npm run e2e` ejecuta Playwright sobre `e2e/` (autenticación, alfabeto e historial) y necesita el backend en marcha.
- **CI.** El repositorio Backend tiene un flujo de GitHub Actions (`.github/workflows/ci.yml`) que compila y prueba cada PR y cada push a `main` y `develop`, con una base PostgreSQL desechable. El repositorio Frontend tiene otro que

comprueba tipos, pruebas, el bundle web, la imagen Docker y las pruebas E2E contra el backend de `main`.

10. Instalación

10.1 Instalación local (resumen)

Ver el Manual de instalación. En resumen, `docker-compose.yml` de la raíz del repositorio Backend levanta 13 contenedores: el gateway, los cinco servicios, las cinco bases PostgreSQL, MongoDB y RabbitMQ. Ningún servicio de la web está en ese compose: el frontend se ejecuta con Expo y apunta al gateway. Cada servicio tiene un `HEALTHCHECK`, y el gateway arranca cuando los cinco servicios responden. `GET /health` devuelve el estado agregado.

10.2 Instalación en servidor web

El archivo `docker-compose.prod.yml` se suma al compose base con `-f`. Quita la publicación del puerto 8080, añade Caddy (puertos 80 y 443, con HTTPS automático mediante Let's Encrypt) y añade el servicio `web`, que se construye desde el repositorio Frontend (ruta `FRONTEND_DIR`, por defecto `../frontendTS`). En producción, el gateway solo recibe tráfico de Caddy y confía en un proxy (`TRUST_PROXY`) para contar las peticiones por IP real.

Requisitos: el dominio apuntando al servidor, y los puertos 80 y 443 abiertos en el servidor y en el panel del proveedor.

Estado del despliegue en `traducesenas.tech`: comprobado el 4 de noviembre de 2026: HTTPS activo, HTTP redirige a HTTPS y `/api/health` responde `ok` con la versión 1.4.0 y los cinco servicios arriba.

11. Ejecución del aplicativo

Ver el Manual de instalación, secciones 2.3 y 3.

12. Real frente a documentado

Tabla 8

Real frente a documentado

Tema	Documentado	Real
Arquitectura	Monolito en documentos antiguos; microservicios como plantilla	Microservicios: gateway y cinco servicios, cada uno con hexagonal interno
Base de datos	PostgreSQL y MongoDB en un solo almacén (ADR-003)	Una base por servicio (ADR-008) y MongoDB para las muestras de gestos
Reconocimiento	MediaPipe con OpenCV; TensorFlow.js	MediaPipe con KNN y DTW, y un modelo propio en TensorFlow.js. Sin OpenCV
Frontend	React Query	Sin React Query
JWT	RS256 en plantillas	HS256 con secreto compartido, caducidad fija y sin renovación
API	`/refresh`, `/logout` y `/jwks` en plantillas	No existen. Rutas reales en la sección 3.2
Migraciones	`node-pg-migrate`	SQL escrito a mano por servicio. Liquibase no se ejecuta
Alcance	Seña a texto o voz	Igual. La tabla `translations` admite además `texto_sena` y `voz_sena`, sin uso en el MVP
Rama de integración	`dev` (convención de los documentos)	`main` es la rama vigente de Backend y Frontend

13. Pendientes

1. Cerrar la sesión de las cuentas bloqueadas: revisar el estado de la cuenta en el middleware compartido o en el gateway (sección 5.4).
2. Decidir si `GET /api/translations/stats` debe exigir `stats.read`.
3. Añadir la regla de fortaleza de contraseña también en el servidor (registro y cambio de contraseña).
4. Crear una pantalla de gestión de roles para administradores (hoy solo hay API).
5. Retirar o sincronizar los changelogs de Liquibase que ya no reproducen el esquema.
6. Contrastar los contratos OpenAPI de `docs/07-api/contracts/openapi` con las rutas de la sección 3.2.

7. Alinear el modelo de referencia (`docs/06-data`) y el ADR-003 con las migraciones y con el ADR-008.
8. Unificar los dos formatos de entrada de la IA y medir la exactitud del modelo.
9. Revisar y rotar las credenciales que hayan quedado en ramas anteriores.
10. Confirmar el estado de HTTPS y de los puertos 80 y 443 en el servidor: comprobado el 4 de noviembre de 2026: HTTPS activo, HTTP redirige a HTTPS y `/api/health` responde `ok` con la versión 1.4.0 y los cinco servicios arriba.

Referencias

Cover, T., y Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, *13*(1), 21–27. https://doi.org/10.1109/TIT.1967.1053964

Expo. (s. f.). *Expo documentation*. Recuperado el 4 de noviembre de 2026, de https://docs.expo.dev/

Google. (s. f.). *TensorFlow.js*. Recuperado el 4 de noviembre de 2026, de https://www.tensorflow.org/js

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales. 17 de octubre de 2012. D.O. No. 48587.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., y Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1906.08172

Meta Platforms, Inc. (s. f.). *React Native*. Recuperado el 4 de noviembre de 2026, de https://reactnative.dev/docs/getting-started

Mora-Zarate, J. E., Garzón-Castro, C. L., y Castellanos Rivillas, J. A. (2025). LSC-54: A landmark-based dataset for Colombian Sign Language. *Data in Brief*, *63*, Artículo 112145. https://doi.org/10.1016/j.dib.2025.112145

OpenJS Foundation. (s. f.-a). *Express 5.x API reference*. Recuperado el 4 de noviembre de 2026, de https://expressjs.com/en/5x/api.html

OpenJS Foundation. (s. f.-b). *Node.js documentation*. Recuperado el 4 de noviembre de 2026, de https://nodejs.org/docs/latest/api/

Provos, N., y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference*. USENIX Association.

Sakoe, H., y Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, *26*(1), 43–49. https://doi.org/10.1109/TASSP.1978.1163055

The PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation*. Recuperado el 4 de noviembre de 2026, de https://www.postgresql.org/docs/16/

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., y Grundmann, M. (2020). *MediaPipe Hands: On-device real-time hand tracking* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2006.10214